

Plan Projektu Klaster EDEN - MVP

Projekt: Koło Naukowe Data Science

7 maja 2026

1 Architektura Systemu - Założenia Ogólne

Zaprojektowana architektura składa się z rozwiązań mających na celu optymalizację pracy klastra. Opiera się ona na dwóch głównych filarach: statycznej analizie kodu przed uruchomieniem oraz monitoringu dynamicznym, który jest powiązany z cyklem życia zadań w systemie Slurm. Monitoring uruchamia się przy starcie zadania i kończy wraz z jego finalizacją.

2 Analiza Statyczna

Zanim zadanie zostanie zatwierdzone, kod użytkownika podlega weryfikacji w celu oszacowania potencjalnych kosztów:

- Do statycznej analizy kodu wykorzystane zostanie narzędzie **Semgrep** z własnymi antywzorcami oraz przypisanymi do nich kosztami.
- Reguły antywzorców będą przechowywane w pliku konfiguracyjnym **YAML**, a Semgrep zwróci zidentyfikowane problemy w formacie **JSON**.
- Wynik zostanie przetworzony przez skrypt w języku **Python**, który obliczy sumaryczny koszt i podejmie decyzję o dopuszczeniu zadania do uruchomienia.

3 Monitoring Dynamiczny w Systemie Slurm

System wykorzystuje dedykowane warstwy do zbierania, przechowywania i analizy danych.

3.1 Warstwa Zbierania Danych

- Proces opiera się na wywoływaniu skryptów **Prolog** i **Epilog** bezpośrednio na węzłach obliczeniowych.
- Zamiast `nvidia-smi`, używamy **NVIDIA Data Center GPU Manager (DCGM)**, który dostarcza metryki z sub-sekundową rozdzielczością przy narzucie CPU poniżej 0,1%.
- Każdy węzeł uruchamia **node_exporter** dla metryk systemowych (CPU, RAM, sieć).

- Skrypt Prolog rejestruje zadanie w katalogu `/run/dcgm-exporter/job-mapping/`, co pozwala automatycznie tagować metryki etykietą `slurm_job_id`. Epilog usuwa ten plik po zakończeniu zadania.

3.2 Warstwa Przechowywania Danych

- Dane składowane są w relacyjnej bazie **PostgreSQL** z rozszerzeniem **TimescaleDB**.
- **Prometheus** pełni rolę bufora czasu rzeczywistego, zbierając metryki co 15 sekund i przechowując je przez 90 dni.
- Retencja (6 miesięcy) oraz downsampling są konfigurowane bezpośrednio w SQL.

3.3 Warstwa Analizy (Serwisy zarządzające)

- Jedynym źródłem prawdy o zadeklarowanych zasobach jest **slurmdbd**.
- Serwis **job_collector** synchronizuje dane przez narzędzie `sacct` co 5 minut.
- Serwis **efficiency_scorer** oblicza wynik efektywności (0-100), a **idle_gpu_monitor** wysyła ostrzeżenia e-mail w przypadku bezczynności GPU.

3.4 Warstwa Prezentacji

Administratorzy korzystają z **Grafana OS** do podglądu stanu klastra i historycznych wykresów.

4 Estymator Zasobów (Docelowa Aplikacja Webowa)

Wprowadza funkcję generowania fragmentu `#SBATCH` na podstawie historii użytkownika

- Użytkownik wybiera framework (np. PyTorch), typ zadania i rozmiar danych.
- System oblicza **medianę** zużytych zasobów z podobnych zadań historycznych.
- Dodawany jest **20% margines bezpieczeństwa** do czasu trwania zadania.

5 Zapewnienie Bezpieczeństwa – Faza Sandbox

Zrezygnowano z wtyczek SPANK oraz `job_submit.lua` ze względu na ryzyko zablokowania klastra w przypadku błędu. Monitoring dynamiczny zostanie przetestowany w **sandboxie**, aby zapewnić stabilność i wydajność rozwiązania.